3. Data Operations

- **2. Arithmetic Operations (Binary)**
  - **Addition:**
    - Addition is the *foundation* of arithmetic operations.
  - **Subtraction:**
    - Subtraction can be transformed into addition.
  - **Multiplication:**
    - Multiplication is a repeated addition operation.
  - **Division:**
    - Division is the inverse operation of multiplication.

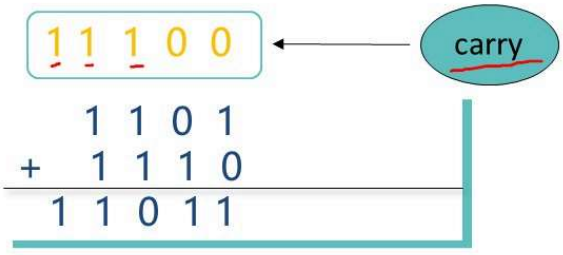
3. Data Operations

- **Addition**

```
  1
  5 4 7
+   2 5
-------
  5 7 2
```

```
  1 1 0
+ 1 0 1
-------
1 0 1 1
```

- sum carry 进位
- 0+0 = 0 0
- 0+1 = 1 0
- 1+0 = 1 0
- 1+1 = 0 1



3. Data Operations

- **Subtraction**
  - **Borrow 1 as 2**

```
  11011
- 10110
-------
    101
```

eg:

```
  1110      14
-  111       7
------
  0111       7
```

3. Data Operations

- **Multiplication**
  - sum carry
  - 0+0 = 0 0
  - 0+1 = 1 0
  - 1+0 = 1 0
  - 1+1 = 0 1

product

0×0 = 0
1×0 = 0
0×1 = 0
1×1 = 1

```
   1010
×   101
-------
   1010
  0000
 1010
-------
 1100010
```

```
   1010
×    11
-------
   1010
 1010
-------
 1110
```

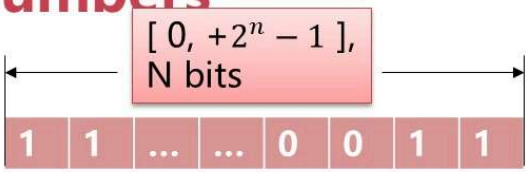
4. Storing Numbers

- **1. Storing integers**

1	2	3
Sign and Magnitude Representation	One's Complement Representation	Two's Complement Representation

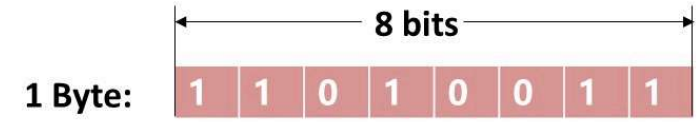
4. Storing Numbers

- **Unsigned Representation**
  - **[0, +∞)**
  - 1. The integer is changed to binary.
  - 2. If the number of bits is less than n, 0s are added to the left of the binary integer so that there is a total of n bits. If the number of bits is greater than n, the integer cannot be stored.



Eg: Store 7 in an 8-bit memory location using unsigned representation.

Change 7 to binary	→					1	1	1
Add five bits at the left	→	0	0	0	0	0	1	1	1

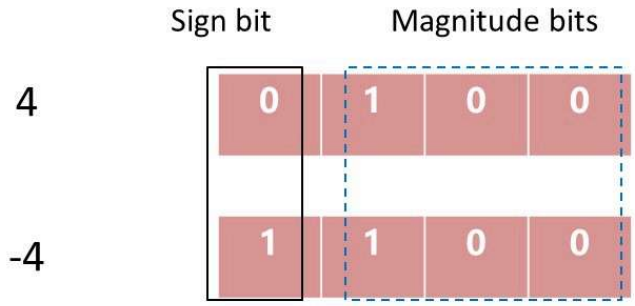


- **Bit – Binary Digit** (smallest unit of data in a computer)
  - **1 Byte = 8 Bits**
    - Enough space for: a single letter or symbol.
  - **1 Kilobyte(KB) = 1024 Bytes = 2^{10} Bytes ≈ 1000 Bytes**
    - Enough space for: about 1/3 page of text.
  - **1 Megabyte(MB) = 1024 Kilobytes = 2^{10} Kilobytes ≈ 1000 Kilobytes**
    - Enough space for: about one book, or one photo, or one minute of music.
  - **1 Gigabyte(GB) = 1024 Megabytes = 2^{10} Megabytes ≈ 1000 Megabytes**
    - Enough space for: about 1000 books, or 1000 photos, or 16 hours of music.
  - **1 Terabyte(TB) = 1024 Gigabytes = 2^{10} Gigabytes ≈ 1000 Gigabytes**
- **PB/EB/ZB/YB...**

01 Sign and Magnitude Representation

4. Storing Numbers

- **Sign and Magnitude Representation**
  - Sign and magnitude representation is a method of encoding signed numbers, where one bit represents the sign and the remaining bits represent the magnitude or absolute value.



Sign and Magnitude Representation

- **Exercises: represent +1, +91, +127, -1, -91, -127, +0, -0, in 8 bits.**

- **Questions: 8 bits, range?**

4. Storing Numbers

- **Sign and Magnitude Representation**
  - **Advantage:**
    - This representation is intuitive and straightforward, using one bit for the sign and the rest for the magnitude, making it easy to understand and implement.
  - **Disadvantage:**
    - Inefficient for computation: Requires additional logic to handle sign bit during arithmetic operations, leading to slower performance.

02 One's Complement Representation

4. Storing Numbers

- **One's Complement Representation**
  - It's a system in which negative numbers are represented by inverting all the bits in the binary representation of their absolute values.
  - To find the one's complement of a negative binary number, simply flip all the bits: 0s become 1s and 1s become 0s.

-4	1 1 0 0	Sign and magnitude
-4	1 0 1 1	1's complement

One's Complement Representation

- **Exercises: represent +1, +91, +127, -1, -91, -127, +0, -0, in 8 bits.**

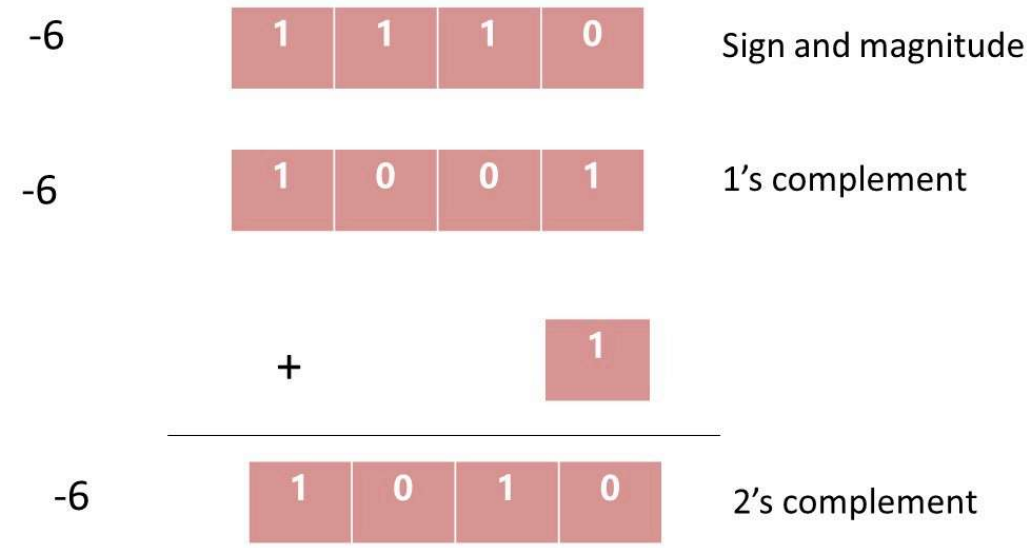
- **Questions: 8 bits, range?**

One’ s Complement Representation

– **Application Scenarios**

- One’s complement is used in digital systems for representing negative numbers, facilitating arithmetic operations and simplifying circuit design.
- In certain computer architectures, one’s complement is used for representing signed integers, allowing for efficient computation and memory storage.

03 Two’ s Complement Representation



Two' s complement → decimal number

Place value	-2^7= -128	2^6=64	2^5=32	2^4=16	2^3=8	2^2=4	2^1=2	2^0=1
digit	1	0	1	1	0	0	0	1
product	-128	0	32	16	0	0	0	1

You now add the values in the bottom row to get -79.

Decimal	Two's Complement(4-bit)
0	0000
1	0001
2	0010
3	0011
4	0100
5	0101
6	0110
7	0111

Decimal	Two's Complement(4-bit)
-8	1000
-7	1001
-6	1010
-5	1011
-4	1100
-3	1101
-2	1110
-1	1111

Two's Complement Representation

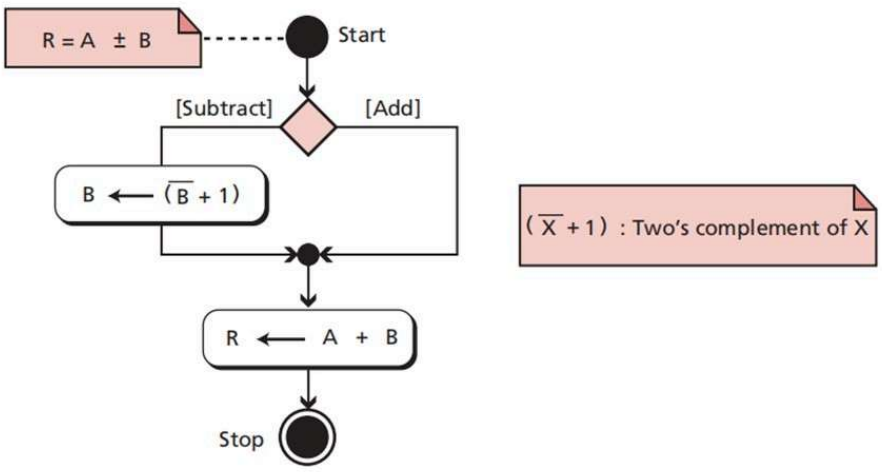
- **Exercises: represent +1, +91, +127, -1, -91, -127, +0, -0, in 8 bits.**

- **Questions: 8 bits, range? (1000 0000B→ -128)**

Summary of integer representations

Contents of memory	*Unsigned*	*Sign-and-magnitude*	*Two's complement*
0000	0	0	+0
0001	1	1	+1
0010	2	2	+2
0011	3	3	+3
0100	4	4	+4
0101	5	5	+5
0110	6	6	+6
0111	7	7	+7
1000	8	–0	–8
1001	9	–1	–7
1010	10	–2	–6
1011	11	–3	–5
1100	12	–4	–4
1101	13	–5	–3
1110	14	–6	–2
1111	15	–7	–1

Binary Arithmetic



Binary Arithmetic

Eg. -9 + 15 = ?

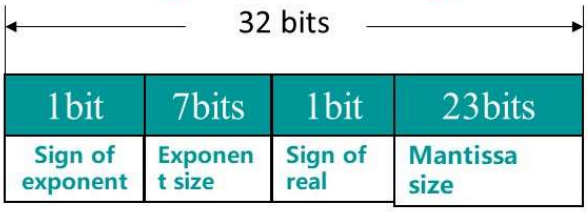
```
  0 0 1 0 0 0 0 1     33
+ 1 0 1 0 0 0 1 0    -94
-------------------------
  1 1 0 0 0 0 1 1    -61
```

Eg. -9 - 6 = ?

- **Question?**
  - **How many numbers can n-bit represent?**

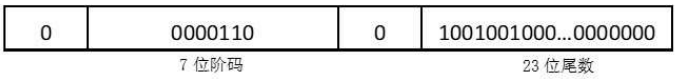
4. Storing Numbers

- **2. Storing reals use Single Precision**



$0.1 \leq |\text{Mantissa}| \leq 1$

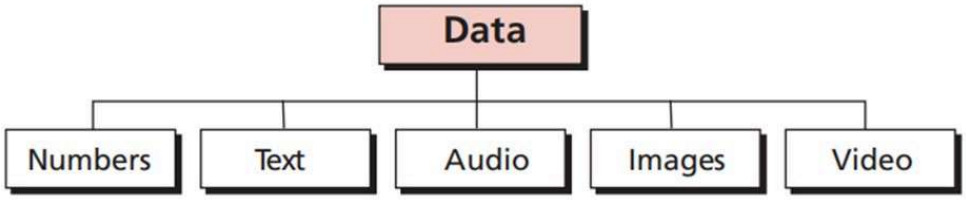
$36.5 = 100100.1B = 0.1001001 \times 2^{6}B = 0.1001001 \times 2^{110}B$



eg: Storing 18.25 or -18.25 in computer?

DATA TYPES

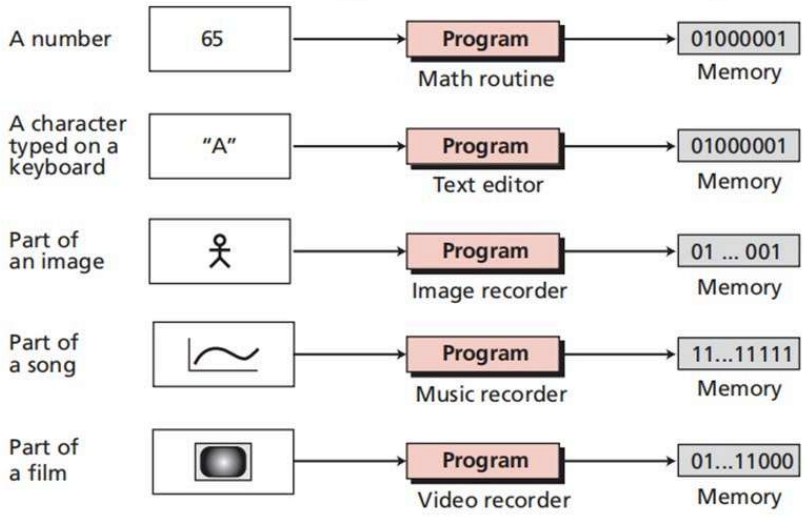
- **Data today come in different forms including numbers, text, audio, images, and video.**



The computer industry uses the term 'multimedia' to define information that contains numbers, text, audio, images, and video.

DATA TYPES

- As Figure shows, a piece of data belonging to different data types can be stored as the same pattern in the memory.



Representing Text

Representing Text

- The general approach for representing characters is to list them all and assign each a binary string.
- To store a particular letter, we store the appropriate bit string.
- Character set: A list of the characters and the codes used to represent each one.
- ASCII is a character set developed in 1906 in an attempt to solve compatibility issues.
- Originally its character codes were 7 bits long, but then it was extended to have a bit length of 8.
  - Original=128 characters, extended=256.

Character code

A ----> 0001

AMERICAN STANDARD CODE FOR INFORMATION INTERCHANGE

Decimal - Binary - Octal - Hex – ASCII Conversion Chart

Decimal	Binary	Octal	Hex	ASCII	Decimal	Binary	Octal	Hex	ASCII	Decimal	Binary	Octal	Hex	ASCII	Decimal	Binary	Octal	Hex	ASCII
0	00000000	000	00	NUL	32	00100000	040	20	SP	64	01000000	100	40	@	96	01100000	140	60	`
1	00000001	001	01	SOH	33	00100001	041	21	!	65	01000001	101	41	A	97	01100001	141	61	a
2	00000010	002	02	STX	34	00100010	042	22	"	66	01000010	102	42	B	98	01100010	142	62	b
3	00000011	003	03	ETX	35	00100011	043	23	#	67	01000011	103	43	C	99	01100011	143	63	c
4	00000100	004	04	EOT	36	00100100	044	24	$	68	01000100	104	44	D	100	01100100	144	64	d
5	00000101	005	05	ENQ	37	00100101	045	25	%	69	01000101	105	45	E	101	01100101	145	65	e
6	00000110	006	06	ACK	38	00100110	046	26	&	70	01000110	106	46	F	102	01100110	146	66	f
7	00000111	007	07	BEL	39	00100111	047	27	'	71	01000111	107	47	G	103	01100111	147	67	g
8	00001000	010	08	BS	40	00101000	050	28	(	72	01001000	110	48	H	104	01101000	150	68	h
9	00001001	011	09	HT	41	00101001	051	29	)	73	01001001	111	49	I	105	01101001	151	69	i
10	00001010	012	0A	LF	42	00101010	052	2A	*	74	01001010	112	4A	J	106	01101010	152	6A	j
11	00001011	013	0B	VT	43	00101011	053	2B	+	75	01001011	113	4B	K	107	01101011	153	6B	k
12	00001100	014	0C	FF	44	00101100	054	2C	,	76	01001100	114	4C	L	108	01101100	154	6C	l
13	00001101	015	0D	CR	45	00101101	055	2D	-	77	01001101	115	4D	M	109	01101101	155	6D	m
14	00001110	016	0E	SO	46	00101110	056	2E	.	78	01001110	116	4E	N	110	01101110	156	6E	n
15	00001111	017	0F	SI	47	00101111	057	2F	/	79	01001111	117	4F	O	111	01101111	157	6F	o
16	00010000	020	10	DLE	48	00110000	060	30	0	80	01010000	120	50	P	112	01110000	160	70	p
17	00010001	021	11	DC1	49	00110001	061	31	1	81	01010001	121	51	Q	113	01110001	161	71	q
18	00010010	022	12	DC2	50	00110010	062	32	2	82	01010010	122	52	R	114	01110010	162	72	r
19	00010011	023	13	DC3	51	00110011	063	33	3	83	01010011	123	53	S	115	01110011	163	73	s
20	00010100	024	14	DC4	52	00110100	064	34	4	84	01010100	124	54	T	116	01110100	164	74	t
21	00010101	025	15	NAK	53	00110101	065	35	5	85	01010101	125	55	U	117	01110101	165	75	u
22	00010110	026	16	SYN	54	00110110	066	36	6	86	01010110	126	56	V	118	01110110	166	76	v
23	00010111	027	17	ETB	55	00110111	067	37	7	87	01010111	127	57	W	119	01110111	167	77	w
24	00011000	030	18	CAN	56	00111000	070	38	8	88	01011000	130	58	X	120	01111000	170	78	x
25	00011001	031	19	EM	57	00111001	071	39	9	89	01011001	131	59	Y	121	01111001	171	79	y
26	00011010	032	1A	SUB	58	00111010	072	3A	:	90	01011010	132	5A	Z	122	01111010	172	7A	z
27	00011011	033	1B	ESC	59	00111011	073	3B	;	91	01011011	133	5B	[	123	01111011	173	7B	{
28	00011100	034	1C	FS	60	00111100	074	3C	<	92	01011100	134	5C	\	124	01111100	174	7C	|
29	00011101	035	1D	GS	61	00111101	075	3D	=	93	01011101	135	5D	]	125	01111101	175	7D	}
30	00011110	036	1E	RS	62	00111110	076	3E	>	94	01011110	136	5E	^	126	01111110	176	7E	~
31	00011111	037	1F	US	63	00111111	077	3F	?	95	01011111	137	5F	_	127	01111111	177	7F	DEL

Left Digit(s) \ Right Digit	ASCII 0	1	2	3	4	5	6	7	8	9
0	NUL	SOH	STX	ETX	EOT	ENQ	ACK	BEL	BS	HT
1	LF	VT	FF	CR	SO	SI	DLE	DC1	DC2	DC3
2	DC4	NAK	SYN	ETB	CAN	EM	SUB	ESC	FS	GS
3	RS	US	□	!	"	#	$	%	&	'
4	(	)	*	+	,	-	.	/	0	1
5	2	3	4	5	6	7	8	9	:	;
6	<	=	>	?	@	A	B	C	D	E
7	F	G	H	I	J	K	L	M	N	O
8	P	Q	R	S	T	U	V	W	X	Y
9	Z	[	\	]	^	_	`	a	b	c
10	d	e	f	g	h	i	j	k	l	m
11	n	o	p	q	r	s	t	u	v	w
12	x	y	z	{	|	}	~	DEL		

- ✓ Characters represented numerically
- ✓ 7 bit encoding(0-127)
- ✓ 65 is LATIN CAPTITAL A

ASCII

- Also note that the first 32 characters in the ASCII character chart do not have a simple character representation that you could print to the screen.
- These characters are reserved for special purposes such as carriage return and tab. These characters are usually interpreted in special ways by what ever program is processing the information.
- Character codes are *grouped* and codes are *sequential*.

Code	character
48	0
49	1
50	2
...	...
57	9
97	a
98	b
...	...
122	z

So if you are given a code for lowercase 'a', you can find the code for lowercase 'e'...

a → e dirrerence of 4 abcd

So e's code is (a's + 4) 97+4=101

In binary the string 'ea' will be encoded as:
$101097_{10} = 11001011100001_2$

...Assuming 7 bit ASCII

ASCII

- E.g. Encode the sentence"This C program has bugs."(without quotes) using the ASCII table (attached at the end of the examination paper). Write down the code for each character in the table as below (in the order from left to right, top to bottom).

ASCII

- E.g. Decode the text represented by the ASCII code given as below, and write it down in the subsequent table:

  01001001 00100111 01101101 00100000 01110111 01110010 01101001 01110100 01101001
  01101110 01100111 00100000 01110100 01101111 00100000 01111001 01101111 01110101
  00100000 01100110 01101111 01110010 00100000 01101000 01100101 01101100 01110000
  00101110

Unicode

- However, extended ASCII is not able to represent all possible characters, so a new character set called Unicode was developed.
- ASCII is a subset of unicode(they share codes up to 127)
- Unicode used 16 bits for each character, but this has since been expanded. It currently has codes for 120,000+characters. It aims to cover all symbols and characters ever used.
- **2^{16} = 65,536 so >> than ASCII**

Pre-Unicode

- In order to decode text, you need to know its encoding
  - Sometimes called code pages(particularly on windows)
  - could be included in a Content-Type header, or the <meta>tags of a HTML page
- What about scripts with thousands of characters?
  - Two-byte encodings for Japanese, Chinese, etc
  - Some encodings used different numbers of bytes for different characters

UNICODE HISTORY

Enter: Unicode

- Unicode is intended to address the need for a workable, reliable world text encoding. Unicode could be roughly described as 'wide-body ASCII' that has been stretched to 16 bits to encompass the characters of all the world's living languages, In a properly engineered design, 16 bits per character are more than sufficient for this purpose.

- Unicode 88 by Joe Becker

Unicode 1.0

- First volume published October 1991
  - 24 scripts including Latin, Thai, Arabic, Hebrew, Cyrillic， etc
- Second volume covering Han ideographs published June 1992
- Any Unicode 1.0 character could be represented in bytes
  - Known as UCS-2 encoding
- Many systems adopted 2 byte character types for strings
  - wchar_t in C90/C++ on Windows*
  - unichar in NSString
  - Java char

An aside - East Asian scripts

- Chinese (hanzi), Japanese (kanji) and Korean (hanja)
  - Logographic scripts
  - All adopted from the writing system used in China during the Han dynasty
- 20,940 character assigned to CJK ideographs in Unicode 1.0 (32%)
- Unicode Han unification effort
  - Allocate one ‘abstract character’ for a single ideograph with on common Han root - even if they are written differently today.
  - saved space, but controversial
  - Text may render differently depending on font choice!

Still ran out of space

- Unicode 2.0 (1996) moved and expanded Hangul, adding 11,172 syllables as new characters
  - Introduced a new way of encoding a larger number of characters using surrogates
  - Unicode 3.1(2001) added new characters outside of 2 byte range
- Expanded remit to cover "all the characters of all the writing system of the world, modern and ancient"
  - 150 scripts
- There are 87,888 CJK Ideographs as of Unicode 12.1
- Emoji

Modern Unicode basics

- Unicode defines a 21-bit codespace of 0x0 – 0x10FFFF(~ 1.1m values)
- Each individual value is known as a *code point*
- Code points are normally written as hex with a U+ prefix, e.g.
  U+0041" A" (LATIN CAPITAL LETTER A)
- The original range of U+0000-U+FFFF is known as the *Basic Multilingual Plane*

UTF - 8

- Variable width, but with a code unit size of 1 byte
- Each code point is encoded in 1-4 code units

Start	End	Byte 1	Byte 2	Byte 3	Byte 4
U+0000	U+007F	0xxxxxxx			
U+0080	U+07FF	110xxxxx	10xxxxxx		
U+0800	U+FFFF	1110xxxx	10xxxxxx	10xxxxxx	
U+10000	U+10FFFF	11110xxx	10xxxxxx	10xxxxxx	10xxxxxx

UTF – 8 Example

- 中

Character	中
Code point	U+4E2D
Binary	0100 111000 101101
UTF-8 Binary	11100100 10111000 10101101
UTF-8 Hex	0xE4 0xB8 0xAD

UTF – 8 Example

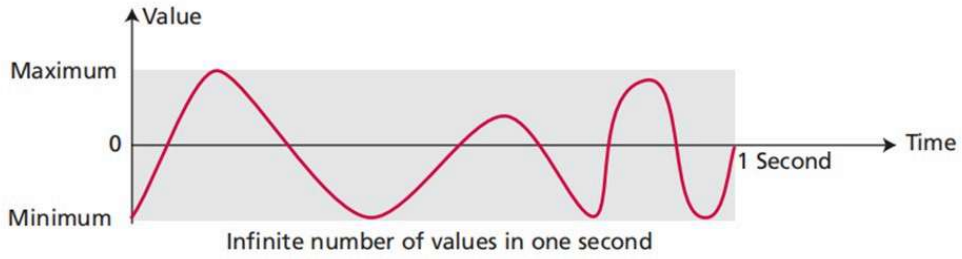
- 

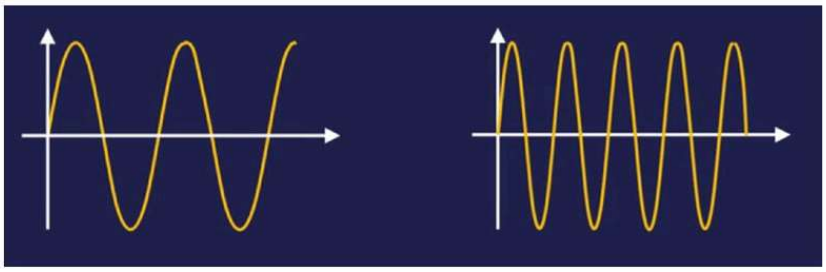
Character	
Code point	U+1F600
Binary	0 0001 1111 0110 0000 0000
UTF-8 Binary	11110000 10011111 10011000 10000000
UTF-8 Hex	0xF0 0x9F 0x98 0x80

Representing Audio

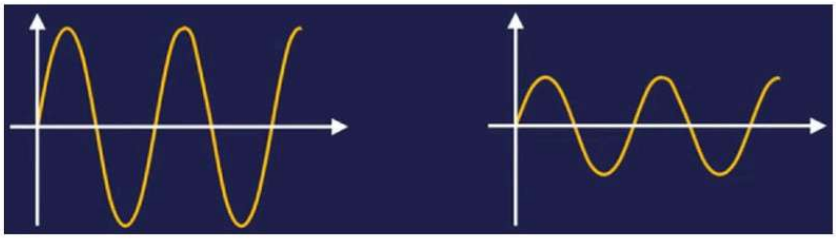
Representing Audio

- Audio refers to sound signals within the frequency range of 15 to 2000 Hz, which are continuously changing analog signals. However, computers can only process digital signals, so it is necessary to convert them into digital signals for the computer to handle. This is the digitization of audio.





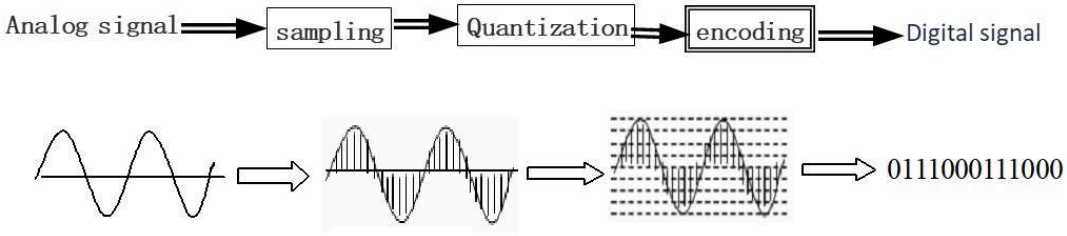
The higher the frequency, the higher the pitch

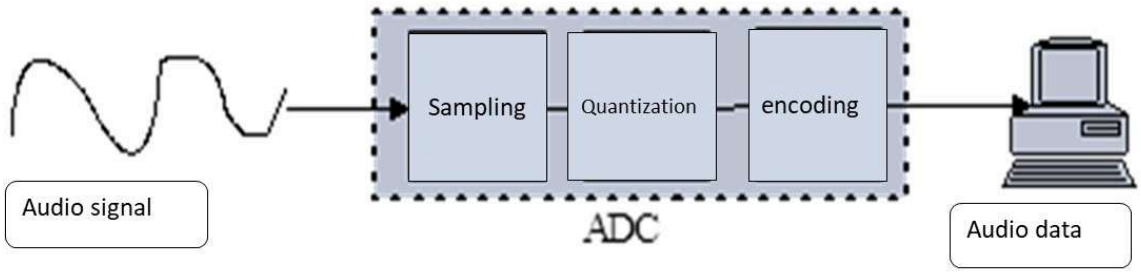


The larger the amplitude, the louder the sound

Representing Audio

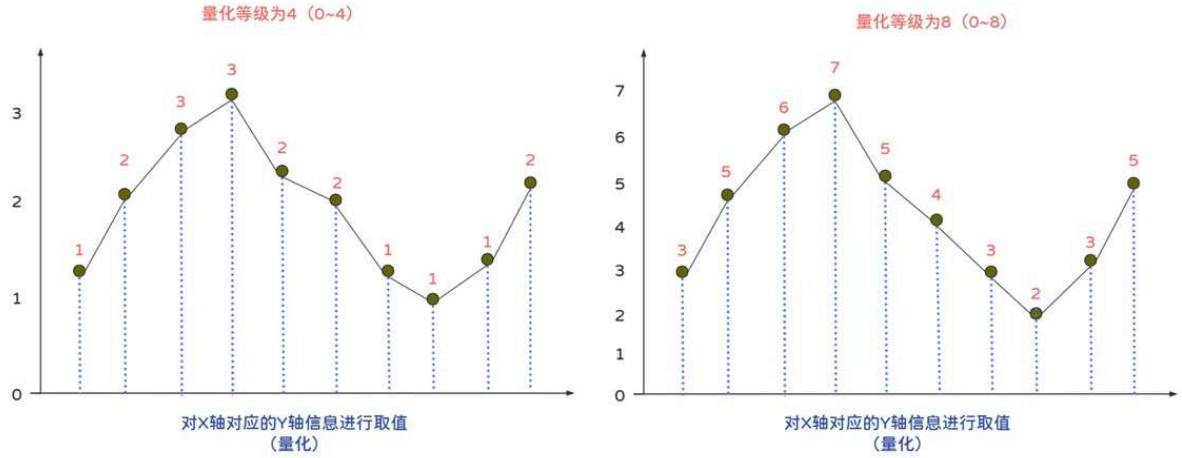
- Audio is not countable. Audio is an entity that changes with time—we can only measure the intensity of the sound at each moment. When we discuss storing audio in computer memory, we mean storing the intensity of an audio signal, such as the signal from a microphone, over a period of time: one second, one hour.





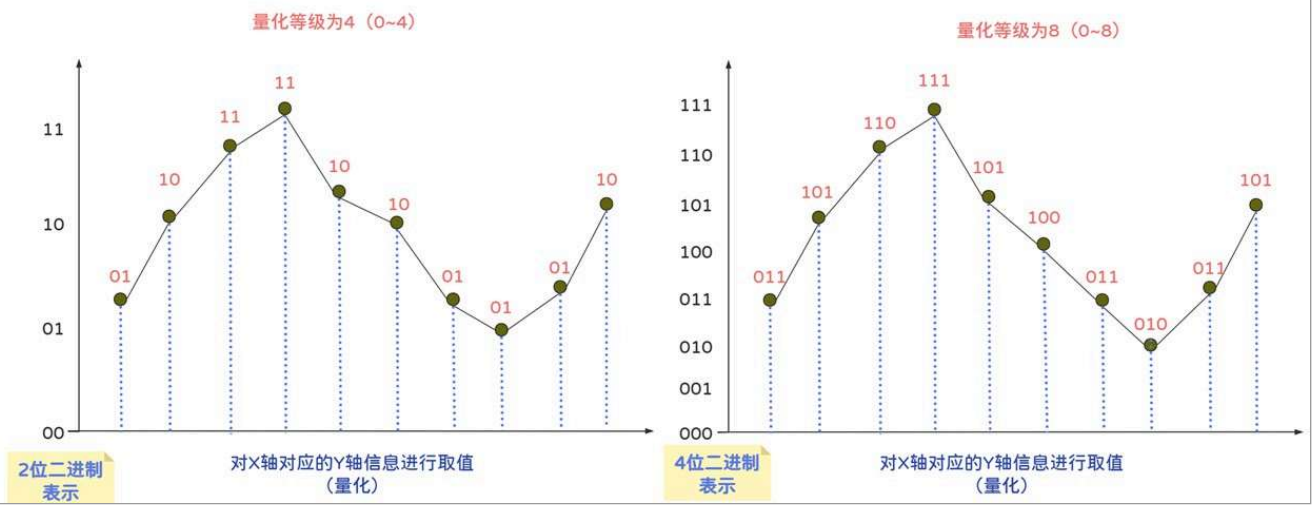
Representing Audio

- **Quantization**
  - Quantization refers to a process that rounds the value of a sample to the closest integer value.



Representing Audio

- **Encoding**
  - The quantized sample values need to be encoded as bit patterns.



Representing Audio

File size(bits)=
Sampling frequency(HZ)*Bit depth(b)*Numbers of Channels*Time(s)

E.g. What is the file size for recording a 1-second stereo audio with a sampling frequency of 44.1kHz and a quantization bit of 16 bits?

44.1Hz*1000*16b/8*2*1s = 176400B

Representing Images and Graphics

- Understand what a pixel is, be able to describe how they relate to an image & the way they are displayer
- Describe the following for bitmaps: size in pixels and colour depth
- Describe using examples how the number of pixels and colour depth can affect the file size of a bitmap image
- Calculate bitmap image file sizes based on the number of pixels and colour depth
- Convert binary data into a black and white image and vice versa

Representing Images and Graphics

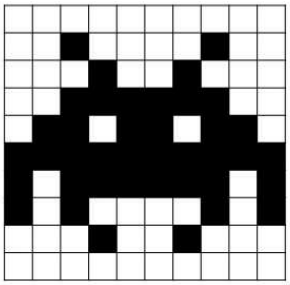
- We see images as analogue, but a computer needs them to be converted to binary to be able to process and store them.
- Images are made up of pixels; these are tiny squares that appear on a screen.
- Each of these pixels has a binary representation.

Pixel - a small dot in an image; many of them together create an image

Representing Images and Graphics

- If an image is simply black and white in colour, then each pixel would be a 1 or a 0

In this image 1 represents black. The binary code for this image would



0000000000
0010000100
0001001000
0011111100
0110110110
1111111111
1011111101
1010000101
0001001000

The size of images

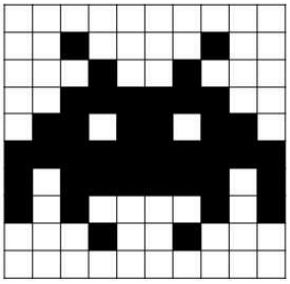
- In order to measure the size of a bitmapped image we need to know two properties-
  - resolution(the number of pixels)
  - the colour depth
- The number of pixels can be found easily by multiplying the number of rows by the number of columns.
- The colour depth can be found by calculating the minimum amount of bits needed to represent every colour.

the size of images

- In this case, we have a 10*10 image, with a colour depth of 1 bit, there is only two colours, so we only need 1 bit to store it. So in total, our image has a size of 100 bits.

To calculate the file size needed to store an image, you use the formula:

file size = number of pixels* space needed to store one pixel

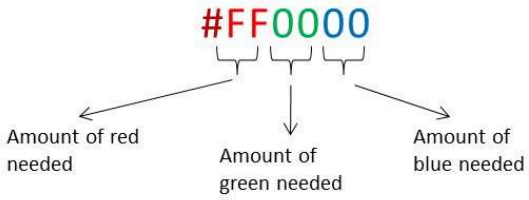


Working out colour depth

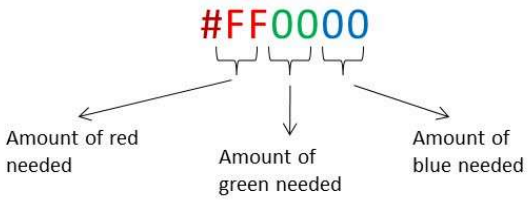
Number of Colours	Amount of bits needed
2 (black and white)	1(0 and 1)
4(Grey scale-Black, dark grey, light gray, white)	2(00, 01, 10, 11)
8(Old 1980s computers)	3(000,001,010,011,100,101,110, 111)
256(Old computers screens)	8(0-255)
16.8M(photo quality)	24(0-(2^24-1))

How can the number of pixels and the colour depth affect the size of an image?

- Most colours are made up of **red**, **green** and **blue**; this is called the RGB colour system.
- The amount of red, green and blue needed to make up the colour will be reflected in the binary data for each pixel.
- These colour codes are shortened to their hexadecimal form so they are easier for us to use.
- An example of a colour code is #FF0000. The code is broken up into the following data:

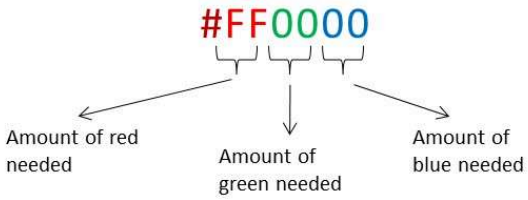


How can the number of pixels and the colour depth affect the size of an image?



- This is the colour code for red; as you can see, it needs the full amount of red but no green or blue.
- If we converted this hexadecimal number to binary it would be
- 111111110000000000000000;
- each colour needed, red, green and blue, is represented by a byte of data.

How can the number of pixels and the colour depth affect the size of an image?



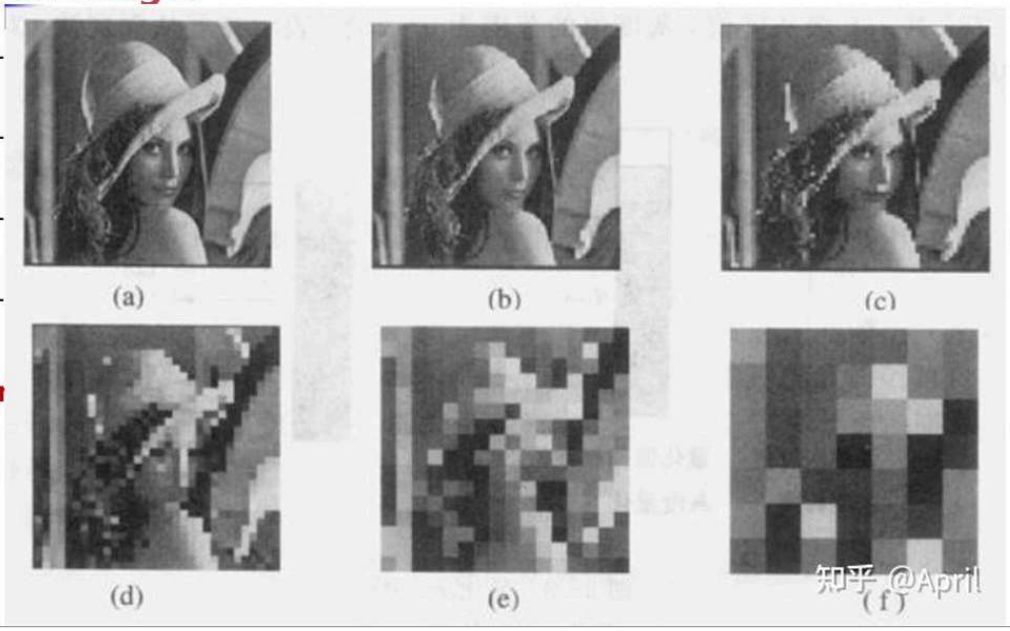
- All this data is needed to create one coloured pixel in an image.
- Thousands of pixels are normally needed to create an image, therefore if each needs at least 24 bits(3 bytes) of data, that' s a lot of data to create an image.

How can the number of pixels and the colour depth affect the size of an image?

- The size of an image file can be affected by the amount of data that needs to be stored.
- The amount of data that needs to be stored is affected by the quality and size of the image.
- If an image has a 2-bit colour depth, it would allow for four different values to be used, 00, 01, 10 and 11, giving four possible colours. The greater the number of bits that are used per pixel, the more colours can be made available.
- The greater the colour depth, the more data needs to be stored to create the image.
- The number of pixels directly affects the physical file size; the more pixels there are, the greater the file size.

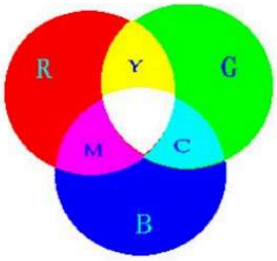
How can the number of pixels and the colour depth affect the size of an image?

- T
- T
- T
- T
- I



Color depth

- Color depth： The amount of data that is used to represent a colour
- Grayscale images: The value of each pixel represents only the intensity information of the light
- TrueColor: A 24-bit color depth(8 bits used for each number in an RGB value)



Exercises

- 1. What is the colour depth of a black and white image?
- 2. If a black and white image is 4 columns across by 8 rows down(resolution is 8*4), how many bytes would it take to store?
- 3. An image has 16 different colours, the resolution is 8*8, how many bytes would it take to store?
- 4.If a black and white image takes 100 bytes to store, how many pixels are there?

Exercises

- 5. If an image has its pixels halved, what happens to the image size and quality?
- 6. If black is 1 and white is 0, draw the following black and white image:{0000,0110,0110,1111}

Standards for image encoding

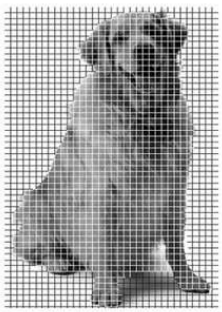
- **BMP(bitmap)**
  - TrueColor color depth, or less to reduce file size
  - Well suited for compression by run-length encoding
- **GIF (indexed color)**
  - File explicitly includes palette of 256 or fewer colors
  - Each pixel thus requires only 8 or fewer bits
  - Animated GIFs are short sequences of images
- **PNG (Portable Network Graphics)**
  - Intended to replace GIFs
  - Greater compression with wider range of color depths
  - No animation

Standards for image encoding

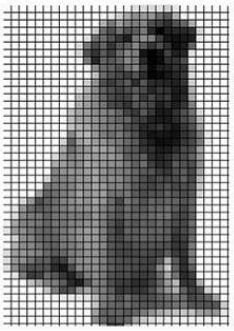
- **JPEG (Joint Photographic Experts Group)**
  - Averages hues over short distances
  - Why? Human vision tends to blur colors together within small areas (science!)
  - How? Transform from the spatial domain to the frequency domain, then discard high-frequency components (math!)
  - Sound familiar? Essentially the same idea used in MP3
  - Adjustable degree of compression

Representing Images

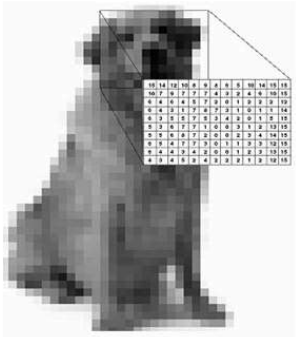
Images



Samplings



Quantization



Digital Images

Vector Graphics

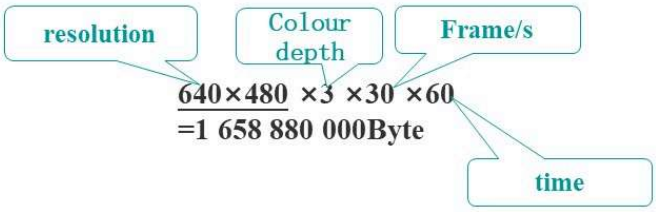
- A format that describes an image in terms of lines and geometric shapes
- A vector graphic is a series of commands that describe shapes using mathematical properties (e.g., direction, length, thickness, color)
- For some types of images, the file sizes can be smaller than with raster graphics because not every pixel is described

- Vector graphics can be resized mathematically and changes can be calculated dynamically as needed.
- Vector graphics are good for line art and cartoon-style drawings.
- Vector graphics are not good for representing images of the real world.

Video

Video is the representation of information that changes in space (single image) and in time (a series of images).

Eg. File size of one minute video



Animation

- Animation is a filmmaking technique whereby still images are manipulated to create moving images.
- Animation is composed of frame by frame images.
- Frame rate(fps): How many frames are squeezed into one second of video.
- In general, the animation playback speed is 12f/s, the video is 25f/s, and what the human eye sees is a continuous picture.